

BIG DATA
ASSIGNMENT #02

M. Hasnain Fareed

F2023376449

A2

Introduction:

Big Data has completely changed how organizations think about data storage and processing. Platforms like social media, banking systems, IoT devices, and healthcare generate enormous volumes of data every single second. Traditional databases simply cannot handle this kind of scale, which is exactly why frameworks like Hadoop were created.

Hadoop is an open-source distributed computing framework maintained by the Apache Software Foundation. It was originally inspired by papers published by Google on their distributed file system and MapReduce programming model. Hadoop solves the big data problem by splitting both storage and processing across many machines working in parallel.

In this assignment, we build a simplified Python simulation of a Hadoop cluster. Instead of setting up actual servers and a full Hadoop installation, we represent Data Nodes as local folders, Name Node metadata as a JSON file, and distributed processing through Python functions. The goal is to understand the internal workings of HDFS and MapReduce from the ground up.

Objective of the Project:

The main goal of this project is to simulate how Hadoop works internally using only Python and basic file system operations.

- Understand how HDFS splits large files into fixed-size blocks
- Implement Name Node and Data Node roles in a simulated environment
- Practice data replication to simulate fault tolerance
- Build a working MapReduce pipeline for data analysis
- Apply a Machine Learning algorithm (KNN) on the processed data
- Connect the concepts of Big Data and Artificial Intelligence

Explanation of Hadoop architecture:

Hadoop has two main parts HDFS (Hadoop Distributed File System) for storage, and MapReduce for processing. Understanding both is essential to grasping how Hadoop can handle petabytes of data efficiently.

HDFS — Hadoop Distributed File System

HDFS is designed for storing very large files reliably across a cluster of machines. When a file is uploaded to HDFS, it is split into blocks of a fixed size (typically 128 MB in production). These blocks are spread across different Data Nodes in the cluster. HDFS provides fault tolerance by keeping multiple copies (replicas) of each block on different nodes, so even if a machine goes down, the data remains accessible.

Name Node

The Name Node acts as the master controller of the entire HDFS. It does not store any actual file data itself; instead, it maintains all the metadata — meaning it knows which blocks belong to which file, where each block is stored, and how many replicas exist. If the Name Node goes down, the entire cluster becomes unavailable,

which is why modern Hadoop setups have a Secondary Name Node or High Availability configuration.

Data Node

Data Nodes are the worker machines that actually store the data blocks. They regularly send heartbeat signals to the Name Node to confirm they are alive. If a Data Node misses too many heartbeats, the Name Node marks it as dead and automatically triggers re-replication of the blocks that were stored on it.

MapReduce

MapReduce is the programming model used for distributed data processing in Hadoop. A job is divided into two phases. In the Map phase, the data is split and processed in parallel across many nodes, producing intermediate key-value pairs. In the Reduce phase, all values for the same key are gathered together (shuffled) and then aggregated into final results. This model is extremely powerful for tasks like counting, aggregation, sorting, and joining large datasets.

Name Node	Master — stores metadata, tracks block locations
Data Node	Worker — stores actual data blocks and replicas
HDFS	Distributed file system with fault tolerance
MapReduce	Distributed processing in Map and Reduce phases
Block	Fixed-size chunk of a file stored in Data Nodes

Replica	Copy of a block stored on a different Data Node
---------	---

Block splitting:

The main.py script reads dataset.csv and splits it into blocks of 5 rows each. Since we have 20 rows, this creates 4 blocks. The blocks are then distributed across three DataNode folders in a round-robin pattern, simulating how Hadoop assigns blocks to different nodes.

```
def split_into_blocks(rows, block_size):
    """Divide rows into fixed-size blocks."""
    blocks = []
    total = len(rows)
    num_blocks = math.ceil(total / block_size)
    for i in range(num_blocks):
        chunk = rows[i * block_size : (i + 1) * block_size]
        blocks.append(chunk)
    return blocks
```

Data Replication:

After storing blocks, the simulation creates one replica for each block on a different Data Node. Replicas are not exact byte-for-byte copies; they include a header line with metadata such as the source node, destination node, block ID, and timestamp. This makes them distinct from the original while still preserving all the data rows.

```
# -----
# PART 2 - DATA REPLICATION (fault tolerance)
# -----

def create_replica(header, block_id, block_rows, primary_node, replica_node):
```

```

    replica_filename = f"{block_id}_replica.csv"
    replica_path      = os.path.join(replica_node, replica_filename)

    replica_header_line = (
        f"# REPLICA | block={block_id} | "
        f"source={primary_node} | "
        f"stored={replica_node} | "
        f"created={timestamp_string()}"
    )

    with open(replica_path, "w") as f:
        f.write(replica_header_line + "\n")
        f.write(header + "\n")
        f.write("\n".join(block_rows) + "\n")

    return replica_filename

def part2_replication(header, blocks, metadata):
    separator("PART 2 – DATA REPLICATION")
    print(f"\nReplication factor: {REPLICATION} (1 original + 1 replica)\n")

    for idx, block in enumerate(blocks):
        block_id      = f"block_{idx:02d}"
        primary_node  = metadata["blocks"][block_id]["primary_node"]
        replica_node  = pick_different_node(primary_node, DATANODES)

        replica_file  = create_replica(header, block_id, block, primary_node,
replica_node)
        metadata["blocks"][block_id]["replicas"].append({
            "node": replica_node,
            "filename": replica_file
        })

        print(f" [REPLICA] {block_id} → {replica_node}/{replica_file}")

    print()
    print("All blocks have at least one replica stored in a different DataNode.")
    return metadata

```

Map reduce:

The mapreduce.py script reads all primary blocks via the Name Node and runs three separate MapReduce tasks:

- Word Count: counts every word appearing in product names across the entire dataset
- Category Frequency: counts how many products belong to each category
- Region Sales: sums up total sales for each geographic region

```
def map_word_count(rows):
    """
    MAP phase: emit (word, 1) for every word in the product 'name' column.
    """
    pairs = []
    for row in rows:
        record = parse_row(row)
        if not record:
            continue
        words = record["name"].lower().split()
        for word in words:
            # Clean punctuation
            word = word.strip(".,!?\\'")
            if word:
                pairs.append((word, 1))
    return pairs

def shuffle_word_count(pairs):
    """
    SHUFFLE phase: group all values by key.
    """
    grouped = defaultdict(list)
    for word, count in pairs:
        grouped[word].append(count)
    return dict(grouped)

def reduce_word_count(grouped):
    """
    REDUCE phase: sum up counts for each word.
    """
```

```

"""
result = {}
for word, counts in grouped.items():
    result[word] = sum(counts)
return result

```

Electronics (category)	7 products
Sports (category)	7 products
Education (category)	6 products
North (sales total)	2150
South (sales total)	545
West (sales total)	495
East (sales total)	317

Machine Learning algorithm:

```

def euclidean_distance(a, b):
    return math.sqrt(sum((x - y) ** 2 for x, y in zip(a, b)))

def knn_predict(train_data, test_point_features, k=3):
    """
    Find the k nearest neighbours in train_data and return the
    majority label.
    """
    distances = []
    for record in train_data:
        dist = euclidean_distance(record["features"], test_point_features)
        distances.append((dist, record["label"]))

    # Sort by distance, take k smallest
    distances.sort(key=lambda x: x[0])
    k_labels = [label for _, label in distances[:k]]

    # Majority vote

```



```
most_common = Counter(k_labels).most_common(1)[0][0]
return most_common
```

Outputs:

Main.py:

```
PS C:\Users\USER\Desktop\Big data\Assignment 2> & C:/Users/USER/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/USER/Desktop/Big data/Assignment 2/main.py"
===== Hadoop HDFS Simulation - Python =====
Started at: 2026-05-16 15:58:22

===== PART 1 - HDFS SIMULATION =====

Reading dataset: dataset.csv
Total rows found: 20
Block size: 5 rows
Blocks created: 4

Distributing blocks across DataNodes...

[STORED] block_00 -> DataNode1/block_00.csv (5 rows)
[STORED] block_01 -> DataNode2/block_01.csv (5 rows)
[STORED] block_02 -> DataNode3/block_02.csv (5 rows)
[STORED] block_03 -> DataNode1/block_03.csv (5 rows)

Total blocks stored : 4
DataNodes used      : DataNode1, DataNode2, DataNode3

===== PART 2 - DATA REPLICATION =====

Replication factor: 2 (1 original + 1 replica)

[REPLICA] block_00 -> DataNode2/block_00_replica.csv
[REPLICA] block_01 -> DataNode1/block_01_replica.csv
[REPLICA] block_02 -> DataNode1/block_02_replica.csv
[REPLICA] block_03 -> DataNode2/block_03_replica.csv

All blocks have at least one replica stored in a different DataNode.

NameNode metadata saved to: namenode.json

===== NameNode Metadata Summary =====

File      : dataset.csv
Total rows : 20
Blocks    : 4
Created   : 2026-05-16 15:58:22

Block ID   Primary Node  Rows  Replica Node
-----
block_00   DataNode1     5     DataNode2
block_01   DataNode2     5     DataNode1
block_02   DataNode3     5     DataNode1
block_03   DataNode1     5     DataNode2

===== DONE =====

Next steps:
python mapreduce.py - run MapReduce word count & category analysis
python ml_model.py  - train and evaluate the ML model

PS C:\Users\USER\Desktop\Big data\Assignment 2>
```

Mapreduce.py:

ML_model.py:

```
===== MapReduce Complete =====

PS C:\Users\USER\Desktop\Big data\Assignment 2> & C:/Users/USER/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/USER/Desktop/Big data/Assignment 2/ml_model.py"
===== PART 4 – Machine Learning Integration =====

Algorithm chosen: K-Nearest Neighbors (KNN)
Features used   : Sales amount, Region (encoded)
Target label   : Product Category

Loading data from DataNodes via NameNode metadata...
Rows loaded: 20
Records ready for ML: 20

Training set : 15 records
Testing set  : 5 records
===== Training Phase =====

KNN is a lazy learner – no explicit training step.
The model stores all training records and classifies
new points by looking at their K nearest neighbours.

K value used : 3
Classes      : Education, Electronics, Sports
===== Prediction Results =====



| Product             | Actual      | Predicted | Result    |
|---------------------|-------------|-----------|-----------|
| USB Hub             | Electronics | Sports    | X Wrong   |
| Badminton Set       | Sports      | Education | X Wrong   |
| Deep Learning Book  | Education   | Sports    | X Wrong   |
| Mechanical Keyboard | Electronics | Education | X Wrong   |
| Swimming Goggles    | Sports      | Sports    | ✓ Correct |



===== Model Accuracy =====

Correctly classified : 1 / 5
Accuracy             : 20.0%

Accuracy is lower because the dataset is very small.
With more data, KNN typically performs much better.

===== ML Complete =====

PS C:\Users\USER\Desktop\Big data\Assignment 2>
```

Results and analysis:

Total dataset rows	20
Block size	5 rows per block
Total blocks created	4
Data Nodes used	3 (DataNode1, DataNode2, DataNode3)
Replication factor	2 (1 original + 1 replica per block)
Total files on disk	8 (4 originals + 4 replicas)
MapReduce tasks run	3 (word count, category, region sales)
ML algorithm	K-Nearest Neighbors (KNN, K=3)
Training / Test split	75% / 25% (15 records / 5 records)
Model accuracy	20% (small dataset limitation)

Challenges faced:

- Ensuring replicas were always stored on a different DataNode from the original block required careful logic to filter out the primary node from the available options.
- Building KNN entirely from scratch — without scikit-learn — meant implementing Euclidean distance, normalization, and majority voting manually, which was a good learning exercise.
- The dataset has only 20 rows, which is intentional for a simulation but makes the ML model perform poorly. In a real Hadoop environment, the same code would process millions of records.
- Reading blocks from DataNode folders while skipping replica header comments required careful line-by-line parsing logic.

- Designing the NameNode JSON schema so it could track both primary and replica locations cleanly took some iteration.

Conclusion:

This project successfully demonstrated the core concepts behind Hadoop's architecture using nothing more than Python and basic file operations. By simulating HDFS, we saw concretely how data is split into blocks, distributed across nodes, and replicated for fault tolerance. The MapReduce implementation showed how large-scale data analysis can be broken into simple map and reduce functions that could theoretically run in parallel across hundreds of machines.

The KNN experiment highlighted an important real-world insight: machine learning models benefit enormously from Big Data. A 20-row dataset cannot give a KNN model enough nearby examples to classify accurately. The same model running on a Hadoop cluster with millions of records would produce much more reliable predictions.

Overall, this assignment provided a hands-on understanding of why Hadoop was a breakthrough technology — it made distributed computing accessible without requiring specialized hardware, enabling organizations to store and process data at a scale that was previously impossible